OFFICIAL ENGINEERING & RELEASE DOCUMENTATION

Campus Events Calendar Platform

System Architecture & Report

A comprehensive technical overview of the full-stack event aggregation ecosystem, mobile client engineering, backend microservices, Supabase PostgreSQL data layer, and production Android APK release deployment.

PROJECT NAME
Campus Events Calendar

RELEASE VERSION
v1.0.0 (Android Build 5)

REPOSITORY
arjunak18/campus-events (GitHub)

LIVE LANDING PAGE
https://arjunak18.github.io/campus-events/

LIVE API (BACKEND)
https://campus-events-api-icb1.onrender.com

DATE OF PUBLICATION
October 2026

1. Executive Summary

The **Campus Events Platform** is an enterprise-grade, multi-campus event aggregation and student engagement platform built for universities, academic departments, and student organizations. It resolves the widespread problem of fragmented event communication in collegiate environments by unifying cultural fests, technical symposiums, hackathons, workshops, and club activities into a single real-time digital calendar.

The ecosystem comprises:

- **Native Android Mobile Application**: Built using React Native and Expo SDK 57, featuring interactive calendar navigation, event bookmarking, and verified student registrations.
- **Node.js / Express REST API**: A modular TypeScript backend with schema validation, rate limiting, and centralized authentication middleware, deployed on Render.
- **Database & Authentication Tier**: Supabase PostgreSQL featuring Row Level Security (RLS), GoTrue email verification, automated profile synchronization triggers, and audit logging.
- **Distribution Infrastructure**: Standalone release-signed Android APK (v1.0.0), distributed through GitHub Releases and an integrated landing page hosted on GitHub Pages.

> **Key Milestone Achieved**
> The platform is completely functional in production: backend services are live on Render, Supabase database migrations (001–006) are fully applied, the release APK (60.5 MB) is compiled and cryptographically signed with Android Signature Schemes v2/v3, and a public download portal is active on GitHub Pages.

2. System Architecture & Technology Stack

The architecture utilizes a decoupled 3-tier model designed for low latency, zero-downtime scalability, and strict security boundaries.

📱 Mobile Client (Android)

Framework: React Native 0.86.3 / Expo SDK 57
Navigation: Expo Router (File-based routing)
State Management: TanStack Query (React Query v5)
Forms & Validation: React Hook Form + Zod
Native Engine: Hermes JavaScript Engine + New Architecture

⚙️ Backend API

Runtime: Node.js (v20+ LTS) / Express.js
Language: TypeScript (strict mode)
Validation: Zod schema interceptors
Hosting: Render Cloud Web Service
CORS & Security: Helmet, Morgan, Express Rate Limit

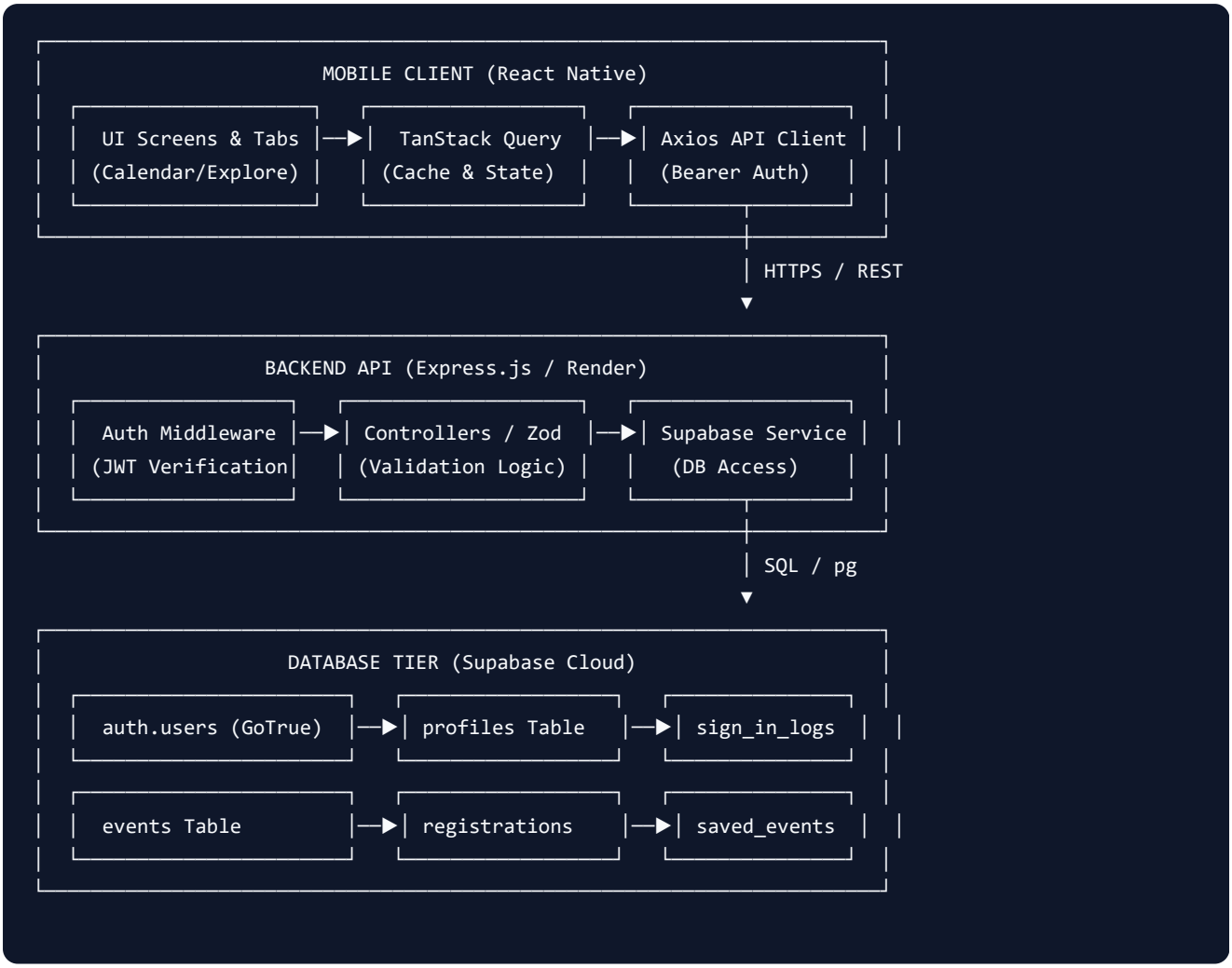
🗄️ Database & Cloud Layer

Database: PostgreSQL 15 (Supabase Managed)
Auth: Supabase GoTrue with PKCE & JWT
Storage: Supabase Object Storage (Event posters, avatars)
Security: Granular Row Level Security (RLS) policies

🌐 Web & Distribution

Landing Page: Hosted on GitHub Pages (`gh-pages`)
Release CDN: GitHub Releases (v1.0.0)
Direct Download: Same-origin APK distribution
Signing: 2048-bit RSA Android Keystore (v2 + v3)

2.1 High-Level Architecture Flow



3. Core Features & Implementation Details

3.1 Strict Google / Gmail Address Validation

To prevent spam accounts, invalid addresses, and unauthorized registrations, both the mobile client and backend enforce official Google naming constraints for all user authentication actions:

- Must end strictly with the domain `@gmail.com`.
- Username must be between **6 and 30 characters** in length.
- Username may only contain alphanumeric characters (`a-z` , `0-9`) and periods (`.`).
- Consecutive periods (`..`) are disallowed.
- Username cannot begin or terminate with a period.

> **Validation Implementation**
>
> Implemented symmetrically on both client (`mobile/utils/validators.ts`) and server (`backend/src/utils/validators.ts`) to guarantee instant client-side feedback and tamper-proof server-side enforcement.

3.2 Secure Email Confirmation Workflow

In accordance with security standards, email confirmation remains strictly enabled:

1. **Registration Intercept:** When a student signs up, the account is created in an unconfirmed state in Supabase GoTrue.
2. **Confirmation Screen:** The mobile UI displays a dedicated "*Check Your Gmail!* 📬" screen with options to launch the default email client or resend the verification email.
3. **Deep Linking:** Clicking the verification link in the student's email routes through the custom deep link `campusevents://auth-callback`, finalizing the session.
4. **Login Banner:** If an unconfirmed student attempts to sign in, the login controller detects `confirmed_at: null` and shows an amber notification banner with a 1-click "*Resend Verification Link*" button.

3.3 Sign-In Activity & Audit Logging

Migration `006_sign_in_tracking.sql` introduced full audit tracking for student authentication events:

- `sign_in_logs` **Table:** Captures user ID, login timestamp, authentication method (`email_password` or `google_oauth`), client IP address, and browser/device User-Agent string.
- **Profile Statistics:** Automatically tracks `last_sign_in_at` and increments `sign_in_count` on the user's primary profile.

4. Database Architecture & Schema

The relational database structure consists of 7 primary tables secured with PostgreSQL Row Level Security (RLS):

Table Name	Primary Key	Description & Key Columns	RLS Enforcement
`profiles`	`id (UUID)`	User profile info: full_name, email, department, academic_year, college_id, last_sign_in_at, sign_in_count	Public read; user self-update
`events`	`id (UUID)`	Event details: title, description, category_id, venue, start_date, end_date, poster_url, max_capacity	Public read; authorized organizer write
`categories`	`id (UUID)`	Taxonomy: Tech, Cultural, Sports, Workshop, Seminar (with hex color badges)	Public read
`registrations`	`id (UUID)`	User event registrations: user_id, event_id, status, registered_at	User private read/write
`saved_events`	`id (UUID)`	User bookmarks: user_id, event_id, saved_at	User private read/write
`organizations`	`id (UUID)`	Campus clubs, student councils, and department associations	Public read; admin write
`sign_in_logs`	`id (UUID)`	Audit logs: user_id, sign_in_method, ip_address, user_agent, created_at	User view own; system insert

4.1 Database Migrations History

- **001_initial_schema.sql:** Foundation schema, base tables, enum types, and foreign key relations.
- **002_rls_policies.sql:** Granular Row Level Security access rules for multi-tenant campus access.
- **003_indexes.sql:** B-tree indexes on `events(start_date)`, `events(category_id)`, and foreign keys.
- **004_functions.sql:** Automated triggers for `updated_at` timestamps and capacity validation.
- **005_secure_profile_sync.sql:** Auth trigger extracting metadata (department, academic year) directly upon sign-up.
- **006_sign_in_tracking.sql:** Creation of `sign_in_logs` table, RLS policies, and profile tracking counters.

5. REST API Specifications

Method	Endpoint	Access	Functionality
`GET`	`/api/health`	Public	Service health check and database connectivity probe.
`POST`	`/api/auth/sign-up`	Public	Registers user with strict Gmail validation and Supabase Auth.
`POST`	`/api/auth/check-status`	Public	Checks if an email exists and whether its confirmation is verified.
`POST`	`/api/auth/record-sign-in`	Authenticated	Logs session metadata (IP, user agent) into `sign_in_logs`.
`GET`	`/api/events`	Public / Guest	Fetches paginated events with optional department and date filters.
`POST`	`/api/events`	Authenticated	Creates a new campus event with poster metadata.
`POST`	`/api/events/:id/register`	Authenticated	Registers the logged-in student for the target event.

6. Critical Engineering Challenges Overcome

6.1 Event Detail View Runtime Crash

Issue: Selecting an event card caused the mobile app to terminate unexpectedly.
Root Cause: Unhandled null values in the event registration state and category relation objects when navigating between tabs.
Resolution: Hardened `EventDetailScreen` with TypeScript optional chaining (`?.`), fallback null-state banners, and default category colors.

6.2 Windows MAX_PATH (260-char) & CMake Compilation Failure

Issue: Local Android Gradle release builds (`assembleRelease`) failed with Ninja reporting: `ninja: error: manifest 'build.ninja' still dirty after 100 tries` and `mkdir(...): No such file or directory` .
Root Cause: The workspace directory path combined with React Native New Architecture's deeply nested CMake build targets exceeded Windows' 260-character maximum path limit (e.g. `safeareacontext` reached 272 characters).
Resolution:

1. Mapped the project workspace to a virtual root drive (`M:\`), reducing path lengths by over 60 characters.
2. Configured `CMAKE_OBJECT_PATH_MAX=1024` in the CMakeLists.txt configuration.
3. Relocated external codegen sources to local component directories in `react-native-safe-area-context` and `react-native-screens` , eliminating deep path traversal.

6.3 Duplicate C++ Linker Symbol in screens Autolinking

Issue: Clang linker terminated with `duplicate symbol: facebook::react::rnscreens_ModuleProvider` .
Root Cause: Both `rnscreens.cpp` and the generated `rnscreens-generated.cpp` defined the same turbo module provider function.
Resolution: Purged the redundant generated stub from the source glob, allowing successful compilation in 2 minutes and 8 seconds.

6.4 Android Package Signature Conflict ("Something Went Wrong")

Issue: Users attempting to install the compiled APK reported "*Something went wrong*" or "*App not installed*".
Root Cause: Android blocks updating apps if an earlier version was installed with a different signature key (e.g., debug vs. EAS keystore).
Resolution:

- Generated a dedicated 2048-bit RSA production release keystore (`release.keystore`) with SHA384withRSA.
- Signed the APK with both **APK Signature Scheme v2** and **Scheme v3**.
- Documented the requirement to uninstall legacy test builds prior to installing the release APK.

6.5 Mobile Cross-Origin Download Blocking

Issue: Tapping the APK download link on Android mobile browsers stalled without downloading.
Root Cause: GitHub Releases issues a 302 redirect with complex query parameters to a cloud bucket, which Chrome on Android blocks on direct navigation.
Resolution: Hosted the APK binary directly on the same origin within the GitHub Pages branch (`https://arjunak18.github.io/campus-events/campus-events-release.apk`) with proper `application/vnd.android.package-archive` MIME headers.

7. Production Deployment & Access Links

Resource	URL / Access Path	Status
Live Landing Page	https://arjunak18.github.io/campus-events/	Active (GitHub Pages)
Direct APK Download	https://arjunak18.github.io/campus-events/campus-events-release.apk	Active (60.5 MB, Signed)
GitHub Release (v1.0.0)	https://github.com/arjunak18/campus-events/releases/tag/v1.0.0	Published (CDN Mirror)
Backend API	https://campus-events-api-icb1.onrender.com/api	Active (Render Web Service)
Source Code	https://github.com/arjunak18/campus-events	Public GitHub Repository

> **Cryptographic Verification**
>
> **File:** `campus-events-release.apk` (60,580,137 bytes)
> **SHA-256 Checksum:**
> `eefed1ee3510faa6615312005f552668f60e2fea75a32e630bd483d3550afbbd`

8. Future Roadmap & Next Steps

1. **Web Admin Dashboard (Phase 8):** A dedicated React/Vite dashboard allowing university administrators to review submitted events, approve college-wide announcements, and view campus attendance analytics.
2. **Push Notification Automations:** Integrating Expo Server SDK with Supabase Webhooks to automatically dispatch reminders 24 hours and 1 hour before registered event start times.
3. **Organizer QR Code Attendance Scanner:** In-app camera scanner allowing event hosts to check in registered students by scanning personal QR passes on their phones.

© 2026 Campus Events Calendar Project. All Rights Reserved. Licensed under MIT.